

Punkt kontrolny 1 — Lista aut się wyświetla

MainPage.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="BazaAut.MainPage">

    <VerticalStackLayout Padding="20" Spacing="15">
    <!-- To główny układ strony.
        VerticalStackLayout układa elementy jeden pod drugim.
        Padding daje odstęp od krawędzi.
        Spacing daje odstęp między elementami. -->

    <Label Text="Baza aut"
        FontSize="24"
        HorizontalOptions="Center"
        FontAttributes="Bold" />

    <CollectionView ItemsSource="{Binding Auta}">
    <!-- To lista elementów.
        ItemsSource="{Binding Auta}" oznacza, że lista ma pobierać dane z kolekcji Auta z C#. -->
    <CollectionView.ItemTemplate>
    <!-- Tutaj zaczyna się szablon pojedynczego elementu listy. -->
        <DataTemplate>
        <!-- DataTemplate mówi, jak ma wyglądać jeden obiekt Auto na ekranie. -->
            <Frame Padding="10" Margin="0,5">
            <!-- Frame to ramka wokół jednego elementu.
                Padding daje odstęp wewnątrz ramki.
                Margin daje odstęp na zewnątrz ramki. -->
                <VerticalStackLayout Spacing="5">
```

```

        <!-- W środku ramki dane są ułożone pionowo.
        Spacing daje mały odstęp między napisami. -->

        <Label Text="{Binding Marka}"
        <!-- Ten napis wyświetla wartość właściwości Marka z obiektu Auto. -->
            FontAttributes="Bold"
            FontSize="18" />

        <Label Text="{Binding Model}"
            FontAttributes="Bold"
            FontSize="18" />

        <Label Text="{Binding Cena, StringFormat='Cena: {0} zł'}"
        <!-- Ten napis wyświetla wartość Cena.
            StringFormat dodaje tekst "Cena:" oraz "zł". -->
            FontSize="16" />

    </VerticalStackLayout>
</Frame>
</DataTemplate>
</CollectionView.ItemTemplate>
</CollectionView>
</VerticalStackLayout>
</ContentPage>

```

MainPage.xaml.cs

```

using System.Collections.ObjectModel;
// To pozwala używać ObservableCollection, czyli kolekcji do przechowywania listy aut.
namespace BazaAut
// To nazwa przestrzeni nazw aplikacji.

```

```

{
    public partial class MainPage : ContentPage
    // To klasa głównej strony aplikacji.
    // Dziedziczy po ContentPage, czyli jest stroną MAUI.

    {
        public ObservableCollection<Auto> Auta { get; set; }
        // To kolekcja aut.
        // Będzie połączona z XAML-em i wyświetlona w CollectionView.

        public MainPage()
        // To konstruktor strony, czyli miejsce, które wykonuje się przy tworzeniu strony.
        {
            InitializeComponent();
            // Ta linijka wczytuje elementy z pliku MainPage.xaml.
            Auta = new ObservableCollection<Auto>
            // Tutaj tworzymy nową kolekcję aut.
            {
                new Auto { Marka = "Toyota", Model = "Corolla", Cena = 65000 },
                // Dodajemy pierwsze auto do listy.
                new Auto { Marka = "BMW", Model = "3", Cena = 120000 },
                // Dodajemy drugie auto do listy.
                new Auto { Marka = "Audi", Model = "A4", Cena = 110000 }
                // Dodajemy trzecie auto do listy.
            };

            BindingContext = this;
            // Ustawiamy źródło danych dla XAML.
            // "this" oznacza: korzystaj z danych z tej klasy MainPage.
            // Dzięki temu XAML widzi kolekcję Auta.
        }
    }
}

```

```
public class Auto
// To klasa Auto.
// Jest jak wzór obiektu, który opisuje jedno auto.

{
    public string Marka { get; set; }
    // Przechowuje markę auta, np. Toyota.

    public string Model { get; set; }
    // Przechowuje model auta, np. Corolla.

    public int Cena { get; set; }
    // Przechowuje cenę auta jako liczbę całkowitą.
}
}
```

W MainPage.xaml.cs tworzymy listę aut i wpisujemy do niej dane.

W BindingContext = this; mówimy XAML-owi: „masz korzystać z danych z tej klasy”.

W CollectionView pokazujemy tę listę na ekranie.

W DataTemplate ustalamy, jak ma wyglądać jedno auto na liście.

Punkt kontrolny 2 — Wybór auta

MainPage.xaml: zmienione i nowe linijki

```
<CollectionView x:Name="Lista"
<!-- Dodajemy nazwę `Lista`, żeby później można było odwołać się do tej kontrolki w XAML. -->
    SelectionMode="Single"
<!-- Włączamy możliwość zaznaczenia jednego elementu z listy. -->
<Label FontSize="18" FontAttributes="Bold">
<!-- Dodajemy nowy napis pod listą. Będzie pokazywał, które auto zostało wybrane. -->
    <Label.FormattedText>
<!-- Zamiast zwykłego tekstu tworzymy tekst z kilku części. -->
        <FormattedString>
<!-- To obiekt, który pozwala zbudować jeden napis z kilku fragmentów. -->
            <Span Text="Wybrałeś: " />
<!-- To stały fragment napisu, który zawsze będzie widoczny. -->
            <Span Text="{Binding Source={x:Reference Lista}, Path=SelectedItem.Marka}" />
<!-- Ten fragment pobiera markę z aktualnie zaznaczonego auta w CollectionView o nazwie `Lista`. -->
            <Span Text=" " />
<!-- To po prostu spacja między marką a modelem. -->
            <Span Text="{Binding Source={x:Reference Lista}, Path=SelectedItem.Model}" />
<!-- Ten fragment pobiera model z aktualnie zaznaczonego auta. -->
        </FormattedString>
    </Label.FormattedText>
</Label>
```

Punkt kontrolny 3 — Panel edycji (Entry)

MainPage.xaml: nowe linijki

```
<Label Text="Edycja wybranego auta"
      FontSize="20"
      FontAttributes="Bold" />

<Entry Placeholder="Marka"
      Text="{Binding Source={x:Reference Lista}, Path=SelectedItem.Marka, Mode=TwoWay}" />
<!-- Tworzymy pole Entry do edycji marki. Binding pobiera Marka z aktualnie zaznaczonego auta w liście. Mode=TwoWay
oznacza, że działa to w obie strony: - dane z auta trafiają do pola - zmiana w polu trafia z powrotem do obiektu. -->

<Entry Placeholder="Model"
      Text="{Binding Source={x:Reference Lista}, Path=SelectedItem.Model, Mode=TwoWay}" />
<!-- Tworzymy pole Entry do edycji modelu. Działa tak samo jak poprzednie, ale dotyczy właściwości Model. -->

<Entry Placeholder="Cena"
      Text="{Binding Source={x:Reference Lista}, Path=SelectedItem.Cena, Mode=TwoWay}" />
<!-- Tworzymy pole Entry do edycji ceny. Entry przechowuje tekst, ale tutaj jest połączone z właściwością Cena.
Dzięki temu użytkownik może wpisać nową wartość ceny wybranego auta. -->
```

Najważniejszy fragment to:

```
Text="{Binding Source={x:Reference Lista}, Path=SelectedItem.Marka, Mode=TwoWay}"
```

- Source={x:Reference Lista} – bierz dane z listy o nazwie Lista
- Path=SelectedItem.Marka – weź właściwość Marka z aktualnie zaznaczonego auta
- Mode=TwoWay – zmiana działa w obie strony

Punkt kontrolny 4 — Suwak/Stepper ceny + podgląd

MainPage.xaml: nowe linijki

```
<Stepper Minimum="0"
          Maximum="300000"
          Increment="1000"
          Value="{Binding Source={x:Reference Lista}, Path=SelectedItem.Cena, Mode=TwoWay}" />
<!-- Dodajemy Stepper, czyli kontrolkę do zmiany liczby przyciskami. Minimum="0" oznacza najniższą możliwą wartość.
Maximum="300000" oznacza najwyższą możliwą wartość. Increment="1000" oznacza, że cena będzie zmieniała się co 1000.
Value jest połączone z ceną wybranego auta. Mode=TwoWay oznacza, że: - Stepper pokazuje aktualną cenę - zmiana
Steppera zmienia cenę w obiekcie Auto. -->

<Label Text="{Binding Source={x:Reference Lista}, Path=SelectedItem.Cena, StringFormat='Aktualna cena: {0} zł'}"
        FontSize="18" />
<!-- Dodajemy Label pokazujący aktualną cenę wybranego auta. Binding pobiera wartość Cena z zaznaczonego auta.
StringFormat dopisuje tekst "Aktualna cena:" oraz "zł". -->
```

Punkt kontrolny 5 — INotifyPropertyChanged (żeby lista odświeżała się od razu)

MainPage.xaml

W punkcie 5 XAML się nie zmienia.

MainPage.xaml.cs: nowe lub zmienione linijki

```
using System.ComponentModel;
// Dodajemy tę przestrzeń nazw, ponieważ zawiera INotifyPropertyChanged
// oraz PropertyChangedEventHandler.

public class Auto : INotifyPropertyChanged
// Klasa Auto zaczyna implementować INotifyPropertyChanged.
// To znaczy: obiekt potrafi poinformować interfejs,
// że jakaś jego właściwość została zmieniona.

private string marka;
// To prywatne pole do przechowywania wartości marki.
// Od teraz nie trzymamy danych tylko w zwykłym { get; set; }.

public string Marka
// To właściwość Marka.
// Będzie korzystała z prywatnego pola marka
// i przy zmianie wywoła informację o odświeżeniu.
{
    get => marka;
// Getter zwraca aktualną wartość z pola marka.
    set
// Setter wykona się wtedy, gdy ktoś przypisze nową wartość do Marka.
```



```
    {  
        marka = value;  
        // Zapisujemy nową wartość do prywatnego pola marka.  
        OnPropertyChanged(nameof(Marka));  
        // Informujemy interfejs:  
        // "właściwość Marka się zmieniła, odśwież miejsca, które ją pokazują".  
    }  
}
```

```
private string model;  
public string Model  
{  
    get => model;  
    set  
    {  
        model = value;  
        OnPropertyChanged(nameof(Model));  
    }  
}
```

```
private int cena;  
public int Cena  
{  
    get => cena;  
    set  
    {  
        cena = value;  
        OnPropertyChanged(nameof(Cena));  
    }  
}
```

```
public event PropertyChangedEventHandler PropertyChanged;
// To zdarzenie przechowuje informację o zmianie właściwości.
// Interfejs nasłuchuje tego zdarzenia.

private void OnPropertyChanged(string propertyName)
// To metoda pomocnicza.
// Będziemy jej używać, żeby zgłaszać zmianę konkretnej właściwości.
{
    PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
// Jeśli ktoś nasłuchuje zmian,
// wysyłamy informację, która właściwość została zmieniona.
}
```

Najważniejsza linijka

```
OnPropertyChanged(nameof(Marka));
```

Po zmianie marki daj znać aplikacji, że marka została zmieniona.

Tak samo działa to dla Model i Cena.

INotifyPropertyChanged jest po to, żeby interfejs od razu widział zmiany danych i automatycznie odświeżał widok.

Punkt kontrolny 6 — Wyróżnienie drogiego auta (DataTrigger)

MainPage.xaml: nowe lub zmienione linijki

```
<Frame Padding="10" Margin="0,5">
<!-- Sama linijka wygląda tak samo, ale teraz do środka Frame dodamy Triggery. -->
```

Te linijki dodajesz **zaraz po otwierającym <Frame ...>**, przed <VerticalStackLayout>.

```
<Frame.Triggers>
<!-- Tutaj zaczynamy sekcję triggerów dla Frame. Trigger to mechanizm: jeśli warunek jest spełniony, wykonaj zmianę.
-->

    <DataTrigger TargetType="Frame"
                  Binding="{Binding Cena}"
                  Value="200000">
<!-- Tworzymy DataTrigger dla Frame. Binding="{Binding Cena}" oznacza: obserwuj właściwość Cena tego auta.
Value="200000" oznacza: zadziałaj wtedy, gdy cena będzie równa 200000. -->

        <Setter Property="BackgroundColor"
                  Value="LightPink" />
<!-- Jeśli warunek jest spełniony, ustaw tło ramki na LightPink. Dzięki temu drogie auto będzie wyróżnione kolorem.
-->

    </DataTrigger>
</Frame.Triggers>
```

Dzięki temu, gdy Cena się zmienia:

- interfejs od razu dostaje informację
- trigger może od razu zareagować
- kolor zmienia się na żywo